Does the better bound win on the clock?

Ben & James / Theory × measurement



A shorter route.

Find shortest paths from one source with non-negative edge weights.

4 → 3

Direct route

Via B: 1 + 2

```
decreaseKey(A, 3)
```

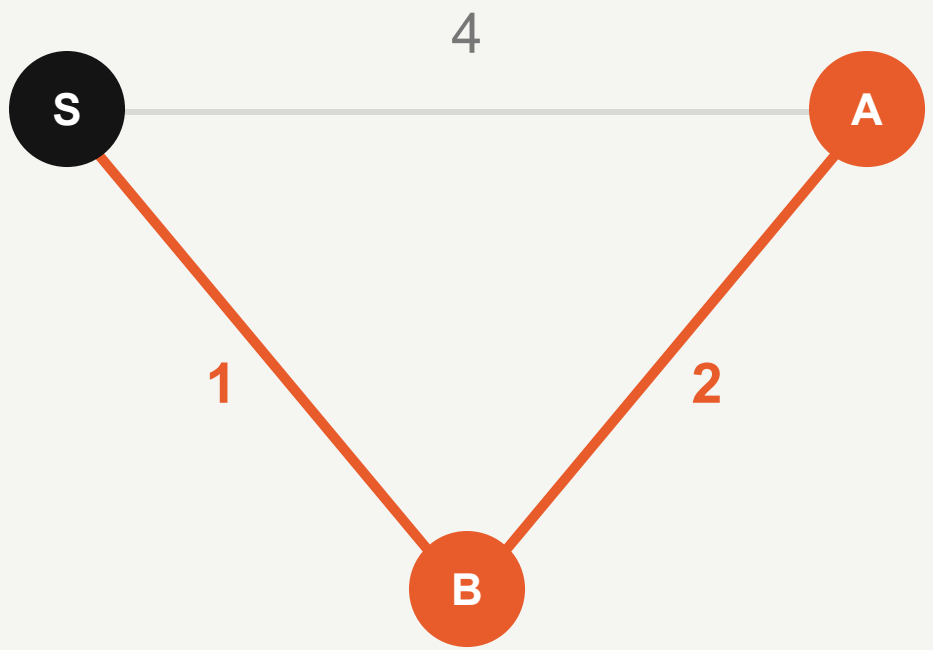


Figure 1. S → B → A costs 3; the direct edge S → A costs 4.

The cost of an update

Compare worst-case binary-heap bounds with amortised Fibonacci-heap bounds.

OPERATION	Binary heap WORST CASE	Fibonacci heap AMORTISED
Insert	`O(log V)`	`O(1)`
Extract minimum	`O(log V)`	`O(log V)`
Decrease-key	**`O(log V)`**	**`O(1)`**
Dijkstra overall	`O((V + E) log V)`	`O(E + V log V)`

Both priority queues use O(V) auxiliary space.

Table 1. Bounds assume adjacency lists and direct queue access. V = vertices; E = edges.

Workload changes the trade-off

Predicted trade-offs before measurement

Lower overhead

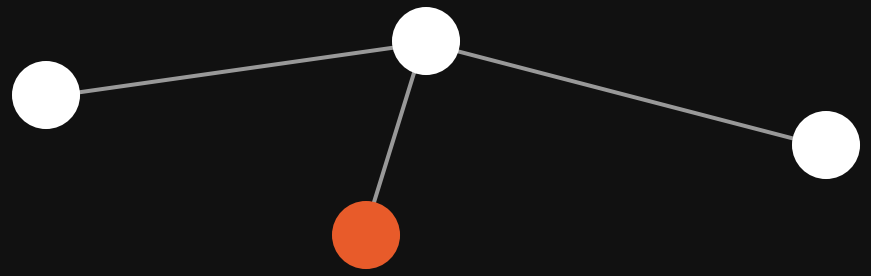
Binary heap: contiguous array



Compact storage may favour binary when queue updates are less frequent.

Cheaper updates

Fibonacci heap: linked nodes



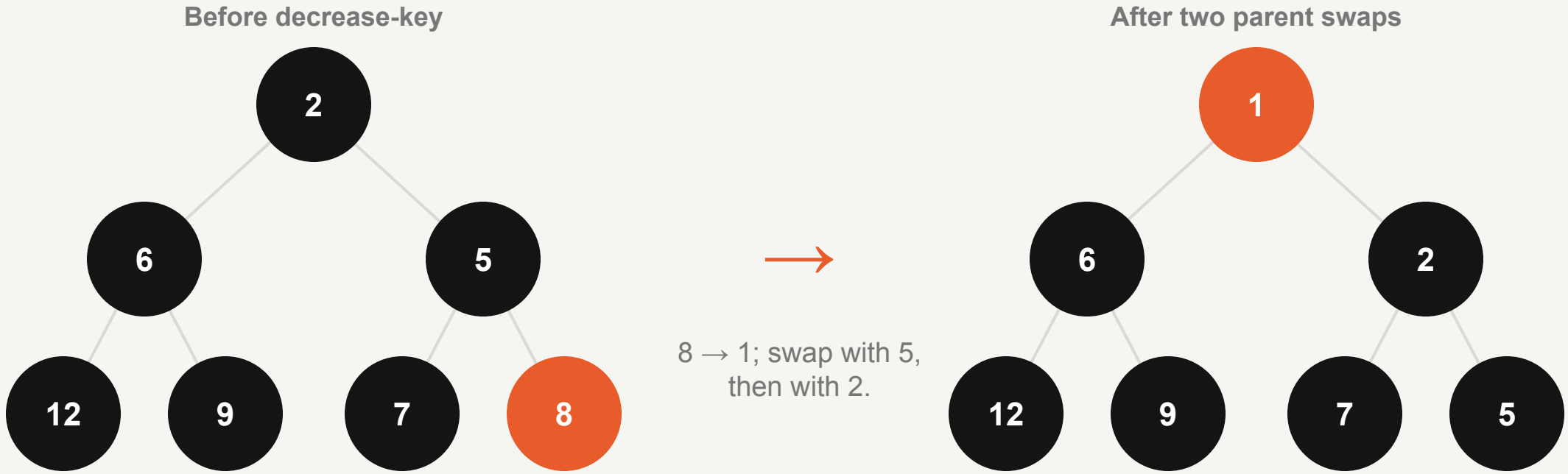
O(1) amortised decrease-key may help when successful updates dominate.

Figure 2. Storage schematics illustrate structure; memory usage was not measured in the matched run.

Binary heap

Heap invariant: a parent’s key is no greater than either child’s key.

James · Binary heap deep dive



`position[vertex]` → O(1) lookup

Sift-up: O(log V)

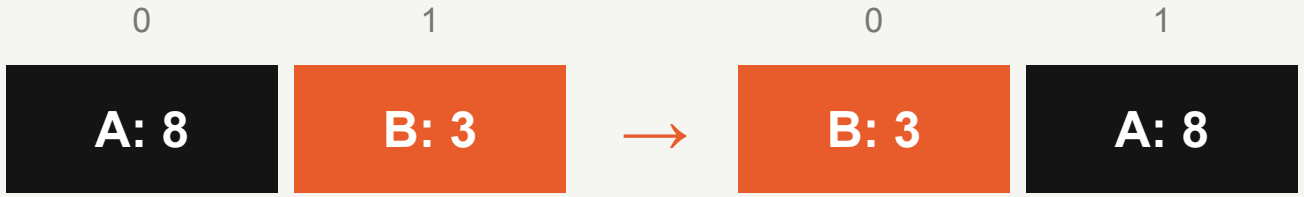
Figure 3. After the key changes from 8 to 1, two parent swaps restore min-heap order.

A swap must update the position map

James · Binary heap deep dive

`Invariant: position[heap[i].vertex] = i`

B: 9 → 3 (before sift-up)




```
position[A] = 0
position[B] = 1
```

After swapping slots 0 and 1


```
position[A] = 1
position[B] = 0
```

```
position[heap[a].vertex] = a;
position[heap[b].vertex] = b;
```

Figure 4. Swapping array entries requires updating both inverse-map entries.

What breaks without it?

A stale position[A] still points to slot 0.

decreaseKey(A, 2) would update B instead.

27

graph datasets

3 sizes × 3 edge ratios
× 3 random seeds

CSV: source, target, weight
Starting vertex: 0

Three graph sizes × three edge-to-vertex ratios



Each cell contains seeds 11, 22 and 33.

Figure 5. Nine configurations × three random seeds give the same 27 inputs to both implementations.

A matched comparison

One machine. Both submitted implementations. The same 27 graphs.

01

Same inputs

Parse CSV files and convert graphs before timing.

→ 02

Both versions

James / binary
Ben / Fibonacci

→ 03

Paired timing

Alternate which version runs first in each trial.

→ 04

Verified outputs

Compare every distance with an independent oracle.

SHARED BUILD

MSVC x64 · C++17 · /O2
Ryzen 5 7500F · Windows 11

2 WARM-UPS / 8 PAIRED TRIALS

```
Vertices:     250 / 1,000 / 4,000
Calls/batch:   32 /     8 /     2
```

Figure 6. Both implementations share the inputs and timing harness; all 27 graphs pass correctness checks.

Fibonacci heap

Lowering the key from 9 to 1 breaks heap order and triggers a cut.

Ben · Fibonacci heap deep dive

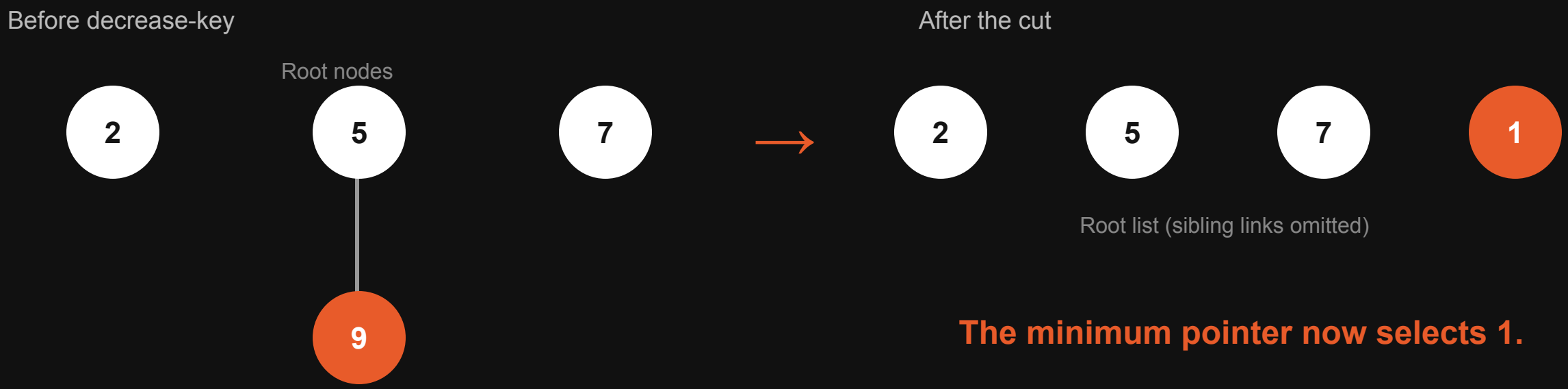


Figure 7. The decreased node joins the root list and becomes the new minimum.

Why a second child loss triggers a cut

Ben · Fibonacci heap deep dive

9 has already lost one child.



Decrease 13 → 1; cut twice.



Cut 1 from 9.
Then cut 9 from 4.
Both become unmarked roots.

First loss
Mark the non-root parent.

Second loss
Cut it; repeat at its parent.

Without the rule
Degree no longer guarantees
a large enough subtree.

Figure 8. Marking and cascading cuts preserve the structural bound used by extract-min.

Matched runtime comparison

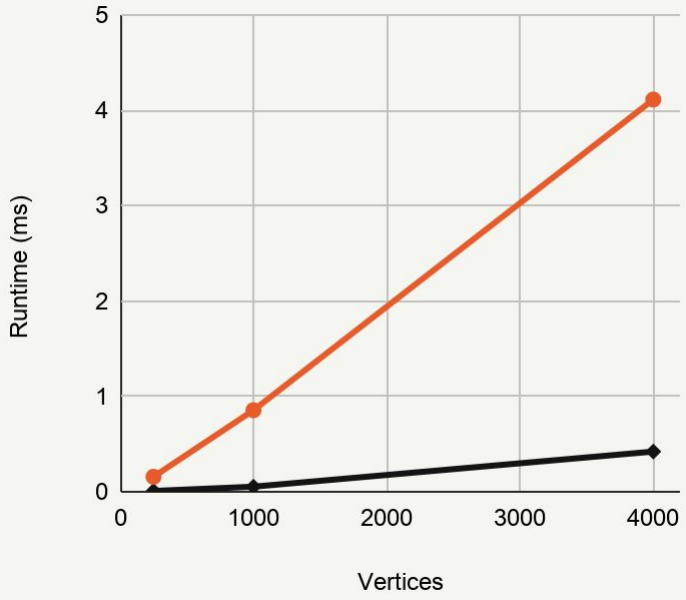
James’s binary implementation has lower medians in all nine configurations.

James’s binary heap

Ben’s Fibonacci heap

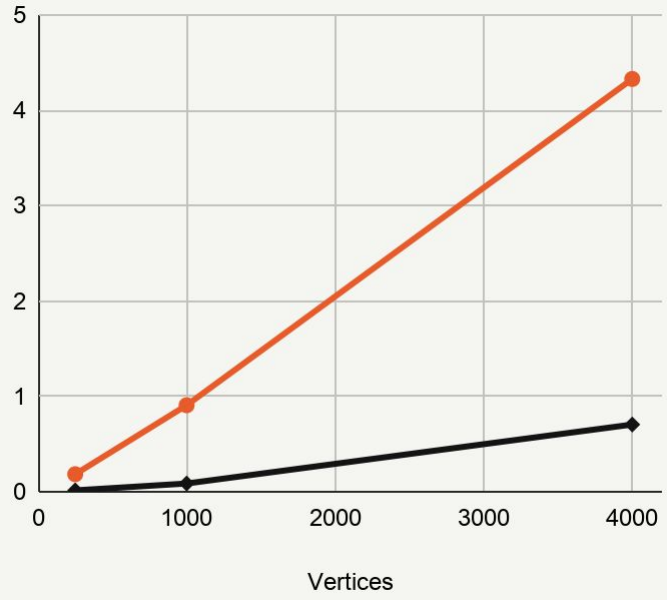
Values below: runtime at V = 4,000

E/V = 4



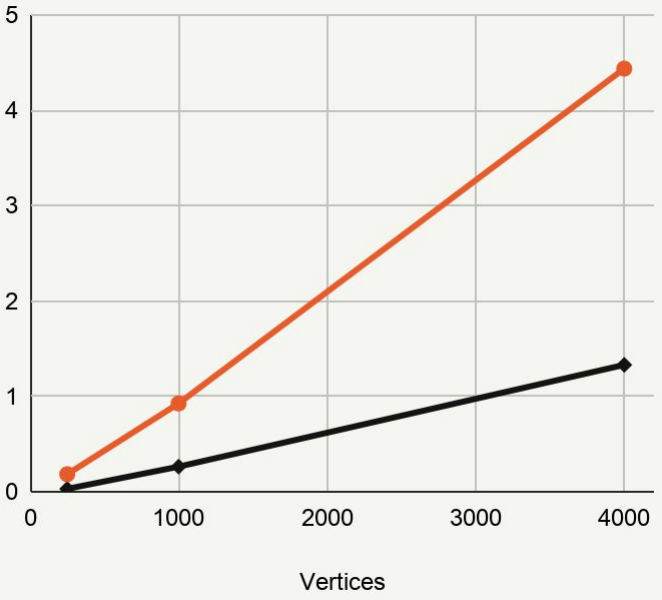
0.422 ms **4.114 ms**

E/V = 16



0.705 ms **4.330 ms**

E/V = 64



1.331 ms **4.438 ms**

Figure 9. Each point is the median of three seed-specific medians. Every panel uses a 0–5 ms scale.

The heap is only part of the difference

Both examine the same edges, but their queue operations and outputs differ.

James / binary heap

Insert vertices when discovered.
Return shortest-path distances.

Ben / Fibonacci heap

Insert all vertices at initialisation.
Return distances and predecessors.

One graph: 10,820 binary decrease-keys vs 14,814 Fibonacci decrease-keys.

All tested families have E = cV. They remain asymptotically sparse; no dense-graph crossover was established.

Example: V = 4,000, E/V = 64, seed 11. Both implementations examine 256,000 edges.

Ben’s surprise: the size of the performance gap

3.33×

Fibonacci runtime ÷
binary runtime

4,000 vertices
E/V = 64

A better decrease-key bound did not translate into a faster complete implementation.

Binary: 1.331 ms



Fibonacci: 4.438 ms



The ratio uses medians across three seeds. A matched implementation comparison, not a heap-only experiment.

9/9

tested configurations

James’s binary implementation was faster.

No crossover in this workload.

Next: use one Dijkstra implementation with interchangeable heaps, then test larger and more update-heavy graphs.

This result applies to these implementations within the tested input range.